

SISTEMA 4:

APLICACIÓN DE CITAS



GRUPO L

GRAGERA SERRADILLA, ALEJANDRO (22M043)

LIU, YANG (23M072)

LÓPEZ GARCÍA, MIGUEL (22M108)

LÓPEZ NÉCULA, ANTONIO JAVIER (23M041)



POLITÉCNICA

Índice

1.	Introducción.....	2
2.	Drivers Arquitectónicos.....	2
	Identificación de actores y casos de uso:	2
	Drivers funcionales.....	3
	Drivers atributos de calidad	4
	Restricciones	5
	Diagramas de secuencia del sistema.....	5
	Diagrama de flujo de Swipe y Match.....	6
	Diagrama de envío de mensaje en chat	6
3.	Interfaces del Sistema (API Rest).....	6
	Recursos.....	7
	Métodos.....	7
4.	Diagrama de la arquitectura.....	9
5.	Componentes de la arquitectura y decisiones de diseño.....	9
	Componentes principales	9
	Decisiones de diseño y trade-offs	10
6.	Anexo: JSON	11

1.Introducción

2.Drivers Arquitectónicos

Identificación de actores y casos de uso:

Actor principal: Usuario (persona que busca conectar con otras).

Actores secundarios:

- Servicio de notificaciones *push* (para enviar alertas).
- Servicio de geolocalización (para obtener ubicación si el usuario lo permite).
- Servicio de autenticación externo (opcional, ej. Google, Facebook).

Casos de uso principales:

- Registrarse en la aplicación: El usuario se registra con email/contraseña o mediante un proveedor externo. Se crea un perfil básico.
- Completar perfil: El usuario añade fotos, intereses, descripción, edad, ubicación, etc.
- Descubrir perfiles: El sistema muestra perfiles de otros usuarios según preferencias y algoritmo de recomendación.
- Deslizar (*swipe*) sobre un perfil: El usuario indica interés (*like*) o rechazo (*pass*) sobre un perfil mostrado.
- Generar match: Cuando dos usuarios se dan «me gusta» mutuamente, el sistema crea un match y notifica a ambos.
- Chatear con un match: Los usuarios con match pueden intercambiar mensajes en tiempo real.
- Recibir notificaciones: El sistema informa al usuario de *likes* recibidos, *matches*, nuevos mensajes, etc.

Drivers funcionales

ID	Driver funcional	Descripción	Motivación
F1	Registro y autenticación	El usuario se debe registrar (email/contraseña o red social) e iniciar sesión de forma segura.	Implica gestión de identidad, tokens JWT, posible integración con OAuth. Fundamental para la seguridad y la personalización.
F2	Gestión de perfil	El usuario puede crear, editar y eliminar su perfil (fotos, intereses, ubicación, etc.).	Requiere almacenamiento de datos variados (imágenes, textos). Impacta en la consistencia y en la latencia de escritura/lectura.
F3	Descubrimiento de perfiles	El sistema debe mostrar al usuario una lista de perfiles candidatos según preferencias (edad, ubicación, intereses) y un algoritmo de recomendación.	Es el núcleo del negocio. Alta complejidad de búsqueda y filtrado. Requiere alto rendimiento y escalabilidad.

F4	Sistema de <i>likes</i> y <i>matches</i>	El usuario puede dar <i>like</i> o <i>pass</i> a un perfil. Si dos usuarios se dan «me gusta» mutuamente, se crea un match.	El usuario puede dar «me gusta» o <i>pass</i> a un perfil. Si dos usuarios se dan «me gusta» mutuamente, se crea un match.
F5	Mensajería en tiempo real	Los usuarios con match pueden intercambiar mensajes. El sistema debe entregar mensajes con baja latencia.	Requiere comunicación bidireccional (<i>WebSockets</i>), persistencia de mensajes, escalabilidad para millones de conversaciones.
F6	Notificaciones <i>push</i> en tiempo real	El sistema debe enviar notificaciones <i>push</i> a los usuarios cuando reciben un <i>like</i> , un match o un nuevo mensaje.	Es clave para la experiencia de usuario y el <i>engagement</i> . Implica integración con servicios externos (FCM, APNs) y requiere un mecanismo asíncrono tolerante a fallos para no bloquear operaciones principales.

Drivers atributos de calidad

ID	Atributo	Escenario
AQ1	Rendimiento (latencia en <i>swipes</i>)	Fuente: Usuario Estímulo: Realiza un <i>swipe</i> (<i>like/pass</i>) Artefacto: Servicio de <i>matching</i>. Entorno: Operación normal, alta concurrencia (picos de 1000 <i>swipes</i>/segundo) Respuesta: El sistema registra el <i>swipe</i> y, si aplica, genera match Medida: El tiempo de respuesta desde que se realiza el <i>swipe</i> hasta que se confirma que la acción es < 200 ms en el percentil 99

AQ2	Escalabilidad (descubrimiento de perfiles)	Fuente: Usuarios concurrentes. Estímulo: Solicitudes de listado de perfiles candidatos. Artefacto: Servicio de descubrimiento. Entorno: Horas punta (ej. 20:00-23:00). Respuesta: El sistema maneja la carga sin degradación notable. Medida: El sistema debe soportar 1 millón de usuarios activos concurrentes manteniendo la latencia < 500 ms para el 99% de las peticiones de búsqueda.
AQ3	Disponibilidad (servicio de <i>matching</i>)	Fuente: Usuarios. Estímulo: Intento de realizar un <i>swipe</i>. Artefacto: Servicio de <i>matching</i>. Entorno: En cualquier momento. Respuesta: El servicio está operativo y procesa la solicitud. Medida: Disponibilidad del 99,99% (tiempo de inactividad planificada < 5 minutos al mes).

Restricciones

ID	Restricción	Descripción
R1	Cumplimiento normativo (GDPR)	El sistema debe cumplir con el Reglamento General de Protección de Datos (GDPR) para los usuarios europeos. Esto implica obtener consentimiento explícito para el tratamiento de datos personales, permitir la portabilidad y el derecho al olvido, y garantizar que los datos sensibles (ubicación, fotos) se almacenen de forma segura. Afecta al diseño del almacenamiento de datos, a los mecanismos de autenticación y autorización, y a los procesos de eliminación de datos.

Diagramas de secuencia del sistema

El proceso de diseño de la arquitectura pide usar diagramas de secuencia de sistema para entender los eventos que ocurren entre el usuario y el sistema. Esto nos guía a la hora de diseñar las APIs.

Diagrama de flujo de Swipe y Match

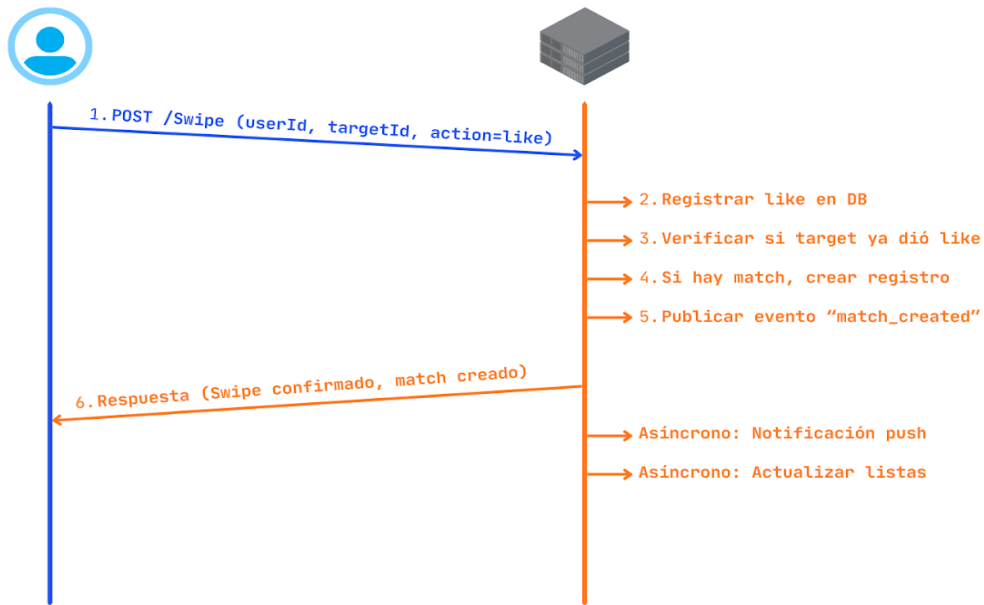
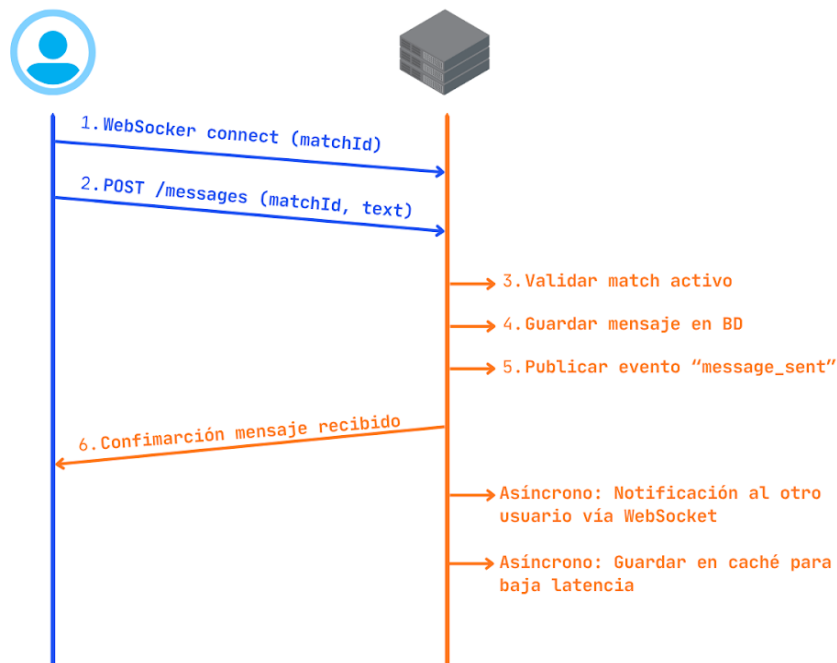


Diagrama de envío de mensaje en chat



3. Interfaces del Sistema (API Rest)

Recursos (incluyendo la información considerada en cada recurso en formato JSON).

Recursos

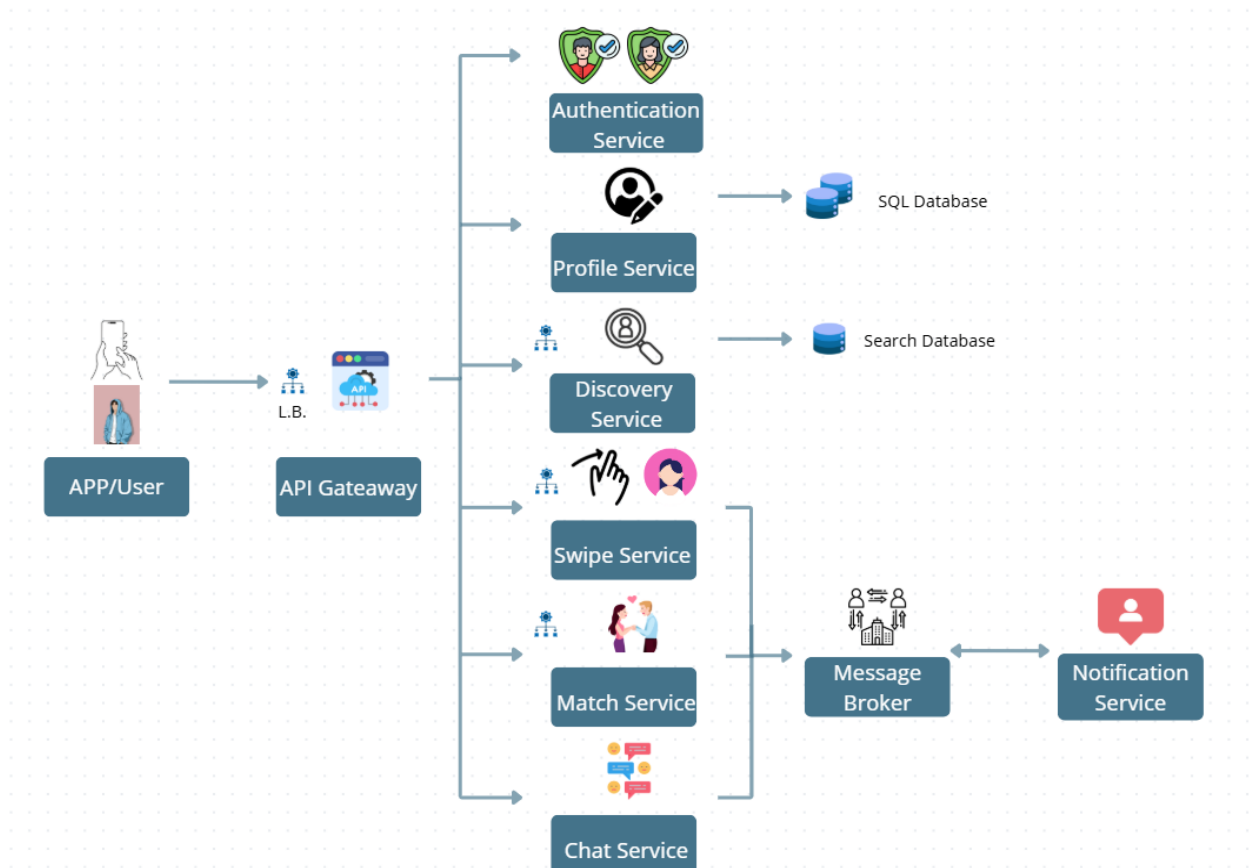
Recursos	URIs
Usuarios	/usuarios/registro /usuarios/login /usuarios/{usuariold}
Perfil	/usuarios/{usuariold}/perfil
Descubrimiento	/descubrimiento
Swipes	/swipes
Matches	/matches /matches/{matchId}
Mensajes	/matches/{matchId}/mensajes
Notificaciones	/usuarios/{usuariold}/notificaciones

Métodos

Recursos	URIs	Métodos	Comentarios
Usuarios	/usuarios/registro /usuarios/login /usuarios/{usuariold}	POST POST GET, DELETE	F1 Registro y autenticación
Perfil	/usuarios/{usuariold} /perfil	GET, PUT, DELETE	F2 Gestión de perfil (fotos, intereses, edad, ubicación)

Descubrimiento	/descubrimiento	GET	F3 Paginación y filtros Debe ser solo lectura, con posibles filtros como /descubrimiento?edadMin=25&ubicacion=Madrid
Swipes	/swipes	POST	AQ1 latencia < 200 ms AQ4 consistencia
Matches	/matches /matches/{matchId}	GET GET	F4 Swipe y match POST /swipe (userId, targetId, action=like) y después se registra like, verificar la reciprocidad, crear match y publicar evento
Mensajes	/matches/{matchId}/mensajes	POST, GET	F5 tiempo real Debe quedar ligado exactamente con el match porque primero se valida que exista match activo
Notificaciones	/usuarios/{userId} /notificaciones	GET	F6 asincronía Solo lectura porque el envío real ocurre de forma asíncrona mediante broker/eventos

4. Diagrama de la arquitectura



5. Componentes de la arquitectura y decisiones de diseño

Componentes principales

- APP/User: Aplicación cliente que interactúa con el sistema. Se limita a la capa de presentación, delegando la lógica de negocio en los servicios *backend*.
- Load Balancer: Distribuye el tráfico entre instancias del sistema, mejorando escalabilidad y disponibilidad. Permite soportar picos de carga típicos de este tipo de aplicación.
- API Gateway: Punto de entrada único que enruta las peticiones a los microservicios. Simplifica la API externa y centraliza aspectos como autenticación o validación. Como inconveniente, puede convertirse en punto único de fallo si no se replica.
- Authentication Service: Gestiona registro, *login* y tokens de autenticación. Se separa por tratarse de una funcionalidad transversal y crítica en seguridad. Usa base de datos SQL para garantizar consistencia (ACID).

- Profile Service: Gestiona la información del usuario (datos personales, preferencias, etc.). Se desacopla de autenticación para mejorar la mantenibilidad.
- Discovery Service: Responsable de mostrar perfiles candidatos. Utiliza Search Database, optimizada para lecturas rápidas. Esta decisión responde a requisitos de rendimiento y escalabilidad, ya que el descubrimiento implica consultas intensivas. El trade-off es la posible consistencia eventual con respecto a los datos de perfil.
- Swipe Service: Registra acciones de *like/pass*. Está optimizado para baja latencia y alto volumen de operaciones.
- Match Service: Gestiona la creación de *matches* cuando existe reciprocidad. Se separa del Swipe Service para garantizar consistencia y evitar duplicados en situaciones concurrentes.
- Chat Service: Permite la mensajería entre usuarios con match. Se implementa como servicio independiente por sus requisitos de tiempo real y baja latencia (p. ej., WebSockets).
- Message Broker: Soporta comunicación asíncrona entre servicios (por ejemplo, creación de match o envío de mensajes). Permite desacoplar componentes y mejorar la disponibilidad, ya que los servicios no bloquean esperando respuestas.
- Notification Service: Consume eventos del *broker* y envía notificaciones *push*. Se desacopla para no afectar al flujo principal del sistema.
- Bases de datos: SQL Database: usada en autenticación y perfiles (consistencia fuerte).
- Search Database: usada en *discovery* (alto rendimiento en lectura). Esta combinación sigue un enfoque similar a CQRS, separando lectura y escritura según sus necesidades.

Decisiones de diseño y trade-offs

- Microservicios: Permiten escalabilidad y mantenibilidad, pero aumentan la complejidad del sistema distribuido.
- API Gateway + Load Balancer: Mejoran desacoplamiento, disponibilidad y gestión del tráfico. A cambio, introducen componentes adicionales que deben ser replicados para evitar fallos.
- Separación de bases de datos (SQL + Search): Optimiza el rendimiento en lectura (*discovery*) sin sacrificar consistencia en operaciones críticas. El coste es la sincronización entre sistemas (consistencia eventual).
- Uso de Message Broker: Permite comunicación asíncrona y mayor tolerancia a fallos. Como inconveniente, aumenta la complejidad y dificulta el seguimiento del flujo de datos.
- Servicio de chat independiente: Adecuado para requisitos de tiempo real. Incrementa el número de servicios, pero permite optimizar específicamente este caso de uso.

6. Anexo: JSON

```
Usuario{
  "id": num,
  "email": String,
  "passwordHash": String,
  "nombre": String,
  "fechaRegistro": String,
  "estado": String
}
Perfil{
  "usuarioId": num,
  "nombre": String,
  "edad": int,
  "descripcion": String,
  "intereses": [String],
  "ubicacion": String,
  "fotos": [String]
}
Descubrimiento{
  "usuarioSolicitante": num,
  "edadMin": int,
  "edadMax": int,
  "ubicacion": String,
  "resultados": [Perfil]
}
Swipe{
  "id": num,
  "usuarioOrigen": num,
  "usuarioDestino": num,
  "accion": String,
  "timestamp": String
}
Match{
  "id": num,
  "usuario1": num,
  "usuario2": num,
  "fechaCreacion": String,
  "estado": String
}
Mensaje{
  "id": num,
  "matchId": num,
  "emisorId": num,
```

```
    "texto": String,  
    "timestamp": String  
}  
Notificacion{  
    "id": num,  
    "usuarioId": num,  
    "tipo": String,  
    "contenido": String,  
    "timestamp": String,  
    "leida": boolean  
}
```